

BACKEND ENGINEERING STANDARD DOCUMENT

Python FastAPI + Git

1. PURPOSE

This document defines **how backend services must be developed**, including:

- Code structure
 - CRUD implementation rules
 - Git workflow standards
 - Review & release process
-

2. TECH STACK STANDARD

Area	Standard
Language	Python 3.11+
Framework	FastAPI
Validation	Pydantic v2
ORM	SQLAlchemy
DB	PostgreSQL / MongoDB
Migrations	Alembic
IDE	Visual Studio Code

Version Control Git + GitHub

Testing Pytest

3. PROJECT STRUCTURE STANDARD (MANDATORY)

```
app/
|
├─ main.py
|
├─ core/
|   ├── config.py           # env variables
|   ├── database.py        # DB connection
|   └─ security.py         # JWT, hashing
|
├─ api/
|   └─ v1/
|       ├── router.py
|       └─ routes/
|           └─ user.py
|
├─ models/
|   └─ user.py
|
├─ schemas/
|   └─ user.py
|
├─ repositories/
|   └─ user_repository.py
|
├─ services/
|   └─ user_service.py
|
├─ utils/
|   └─ exceptions.py
```

```
|   └─ constants.py
|
├─ tests/
|   └─ test_user.py
|
└─ requirements.txt
```

- ✗ No flat files
 - ✗ No business logic in routes
 - ✗ No DB queries outside repositories
-

4. LAYER RESPONSIBILITY RULES

Layer	Allowed Responsibility
Router	HTTP, status codes
Service	Business logic
Repository	Database access
Schema	Validation & serialization
Model	Table structure

5. NAMING CONVENTIONS

Item	Rule
File names	<code>snake_case.py</code>
Class names	<code>PascalCase</code>
Functions	<code>snake_case()</code>

Constants `UPPER_SNAKE_CASE`

Routes plural nouns

6. CRUD DEVELOPMENT STANDARD

Every CRUD must implement:

Operation	HTTP	Path
Create	POST	<code>/resource</code>
Read	GET	<code>/resource/{id}</code>
List	GET	<code>/resource</code>
Update	PUT	<code>/resource/{id}</code>
Delete	DELETE	<code>/resource/{id}</code>

7. EXAMPLE TASK – USER CRUD

7.1 User Table Definition

Field	Type
id	int (PK)
name	string
email	string (unique)
isActive	boolean

createdAt datetime

7.2 Mandatory Files for User CRUD

```
models/user.py
schemas/user.py
repositories/user_repository.py
services/user_service.py
api/v1/routes/user.py
tests/test_user.py
```

Each file **must exist**.

8. ERROR HANDLING STANDARD

- No raw exceptions
- No DB errors exposed
- Custom exceptions only

```
raise NotFoundException("User not found")
```

9. GIT STANDARD (MANDATORY)

9.1 Branching Strategy

```
main           → production
dev            → integration
feature/*      → new feature
bugfix/*       → bug fix
hotfix/*       → production fix
```

9.2 Branch Naming Rules

feature/user-crud

bugfix/user-email-validation

hotfix/login-failure

10. COMMIT MESSAGE STANDARD

Format (MANDATORY)

<type>: <short description>

Allowed Types

Type	Usage
feat	New feature
fix	Bug fix
refactor	Code improvement
test	Tests
docs	Documentation
chore	Config / infra

Examples

feat: add user CRUD APIs

fix: handle duplicate email error

refactor: move db logic to repository

✗ update code

✗ final commit

✗ changes

11. PULL REQUEST (PR) STANDARD

PR Title

feat: user CRUD implementation

PR Description (Mandatory)

- Implemented User CRUD APIs
- Added validation schemas
- Added repository & service layers
- Added unit tests

PR Checklist

- Code follows STD
- No hardcoded values
- Tests added
- Lint passes
- No unused files

12. CODE REVIEW RULES

PR must be rejected if:

- Business logic in routes
- DB queries in service
- No tests

- STD violations
 - Poor naming
-

13. RELEASE STANDARD

Step	Rule
Merge	feature → dev
QA	dev tested
Release	dev → main
Tag	v1.0.0

14. VERSIONING (SEMVER)

MAJOR.MINOR.PATCH

Change	Version
Breaking	MAJOR
Feature	MINOR
Bug fix	PATCH

15. SECURITY RULES

- Secrets only in .env
- Never commit .env

- JWT expiry mandatory
 - Password hashing required
-